BACKEND THE LAST PART :

Install.ps1: this is a PowerShell installation script that automatically sets up all the dependencies for our dz carpooling backend on windows and automates the entire setup process, so we don't have to install packages manually one by one

```
Start Script
    ↓
1. Check if Python is installed
    ↓
2. Update pip (package installer)
    ↓
3. Install critical packages first
   (Django, DRF, PostgreSQL driver)
    ↓
4. Install Pillow (image processing)
    ↓
5. Install all other packages from requirements.txt
    ↓
6. Verify everything is installed correctly
    ↓
7. Show next steps to start the project
```

Without Script	With Script
Type 20+ pip commands manually	Run one script
Remember exact package versions	Versions handled automatically
Debug installation issues	Automatic verification
Takes 10-15 minutes	Takes 2-3 minutes

```
# 1. Open PowerShell as Administrator
# 2. Navigate to your project folder
cd C:\Users\You\DZ-CarPool\backend

# 3. Run the installation script
.\scripts\install.ps1

# 4. Follow the next steps shown at the end
```

Pricing.py: this file provides helper functions for working with fuel prices extracting wilayas from locations and calculating suggested trip prices based on fuel costs and it simulates an external API for fuel prices and calculates how much a trip should cost based on fuel consumption

Function	Purpose
`load_fuel_prices()`	Load external price data
`extract_wilaya_from_location()`	Convert city names to regions
`get_fuel_price_for_wilaya()`	Get price by region
`get_average_fuel_price()`	Fallback for missing data
`get_default_consumption()`	Get average fuel consumption
`calculate_suggested_price()`	**Main function** - calculates recommended price
`get_fuel_prices_summary()`	Returns all data for API endpoints

Search_utils.py: this file contains intelligent matching logic that finds the best trip for users based on city name dates and regions even when users spell cities differently and it helps for matching cities even with typos or alternate names, finding trips in nearby regions if exact matches aren't available, scoring how well a trip matches a user's search

Function	Purpose	Input	Output	Example
`normalize_city_name()`	Standardizes city names (removes accents,	City string	Normalized string	`"Béjaïa"` → `"bejaia"`

	lowercase)			
`get_city_similarity()`	Calculates how similar two city names are (0-1)	city1, city2	Float (0.0-1.0)	`"Alger"` & `"Algiers"` → `0.95`
`get_regional_proximity()`	Scores how close two cities are geographically	city1, city2	Float (0.3-0.7)	Same region → `0.7`
`calculate_date_proximity()`	Scores how close two dates are	target_date, trajet_date	Float (0.1-1.0)	Same day → `1.0`
`calculate_match_score()`	**Main function** - calculates overall trip match score	search params + trip params	(total_score, details)	Perfect match → `1.0`
`get_time_period()`	Converts time to time of day	hour string	"morning"/"afternoon"/"evening"	`"14:30"` → `"afternoon"`

Function	Use Case
`normalize_city_name()`	Prepare city names for comparison
`get_city_similarity()`	Match "Alger" with "Algiers" or "Alger Centre"
`get_regional_proximity()`	Show trips from nearby regions when exact matches aren't available
`calculate_date_proximity()`	Show trips within a few days of user's preferred date
`calculate_match_score()`	Rank trips by relevance (best matches first)
`get_time_period()`	Filter trips by morning/afternoon/evening

This algorithm **finds the best matching trips** by comparing cities (handling typos, alternate names, and regions) and dates scoring each trip from 0-100% so users see the most relevant results first

.dockerignore : this file tells docker witch filles and folders to ignore when building our docker image and when an image been build with docker, docker sends all files from our project folder to the docker daemon and this file prevents unnecessary files from being included

Gitignore: this file tells git witch files and folders to ignore when committing code to our repository and keeps our git clean by excluding local caches virtual environments so they never get exposed on git hub